Getting started with clusterRCTsim

Jonathan Che

2023-08-24

Overview

This document walks through running simulations for the clusterRCT project, both locally and on the FASRC cluster.

Running simulations locally

This section covers how to run a single set of simulations locally.

> TL;DR: source `estimator_performance_simulation.R` to run a simulation and source `data_prep/fully_prepare_simulation_results.R` to clean up and process the results of the simulation

File structure

- `simulation_support_code_crt.R`: contains functions for running simulation
- `estimator_performance_simulation.R`: script for running simulations
- `data_prep` folder
  - `fully_prepare_simulation_results.R`: calls the three scripts below, in order
  - `bundle_odyssey_results.R`: aggregates all results files
  - `clean_simulation_results.R`: cleans aggregated output
  - `prepare_aggregate_results.R`: produces nice outputs
- `analysis` folder
  - `calculate_paper_results.Rmd`: basic summaries of simulation results
  - `designing_simulation2.Rmd`: (don't try to source this file) random collection of a bunch of explorations of different simulations

How to run a set of local simulations

As described in the TL;DR, source `estimator_performance_simulation.R` to run a simulation. The specific list of simulation scenarios is picked by the `group` variable at the top of `estimator_performance_simulation.R`, which is passed to the function `make.scenario.list()` (in `simulation_support_code_crt`), which finally makes the full list of simulation settings that are run in a single simulation run.

After running the simulation, source `data_prep/fully_prepare_simulation_results.R` to compile the results. These results are stored in `results/aggregated_simulation_results.RData`.

Loading this file produces a lot of objects in your environment to explore. I'll highlight `sruns`, which is a nice big summary table of metrics for each simulation scenario.

Other notes

- The simulation runs `NUM_TRIALS_PER_SCENARIO = 4` runs of all given simulation scenarios.

- When running things locally, `Sys.getenv("SLURM_ARRAY_TASK_ID")` will return an empty string. This causes things to run in "testing mode" which subsets the provided scenarios.
  - If you want to run all scenarios locally: in R, run `Sys.setenv("SLURM_ARRAY_TASK_ID" = 1)`
  - The results of the simulation will then be saved as `results/simulation_results_1.rds`

Running simulations on the cluster

This section assumes that you are all set up with the FAS RC research cluster. If you aren't, check the 'Getting Started' section in the link below:

`https://docs.rc.fas.harvard.edu/`

Files

- `install_needed_packages.R`: (potentially non-comprehensive) listing of packages needed to install on Odyssey to get the simulation to work
- `test_it.slurm`: runs 2 replications to test things
- `run_it.slurm`: runs full simulation (98 more replications)
- `results.slurm`: runs fully_prepare_simulation_results.R (to run after `run_it.slurm` finishes)

How to run a set of simulations on the cluster

To set things up:

1. Upload the required files to the cluster (using FileZilla or however you usually do it). The required files are:
   - `estimator_performance_simulation.R`
   - `simulation_support_code_crt.R`
   - `data_prep/bundle_odyssey_results.R`
   - `data_prep/clean_simulation_results.R`
   - `data_prep/prepare_aggregate_results.R`
   - `data_prep/fully_prepare_simulation_results.R`
2. Install the required packages in `install_needed_packages.R` however you typically do that (See below for a primer if you're out of practice)
3. Make a `logs` directory under clusterRCTsim (for log files, see below)

To run the simulation:

1. In the cluster command line, run: `sbatch test_it.slurm` to check that everything works
   - To be nice, run: `sacct -u your_username --format=jobname,state,maxrss,reqmem,elapsed`
   - This shows how much memory and time the test took so you can adjust `run_it.slurm` to ask for only a bit more than that. Because we're running all scenarios in each job, all replications should require similar resources, which is nice.
2. Run `sbatch run_it.slurm` to run full simulation
3. Run `sbatch results.slurm` to aggregate simulation results
4. Download `results/aggregated_simulation_results.RData` using FileZilla or however you usually do it.

Notes for `run_it.slurm`

- The slurm script that user has packages installed in `apps/R-4.2.2` directory on Odyssey. Change this line if you have packages installed elsewhere.
  - See https://docs.rc.fas.harvard.edu/kb/r-packages/
- The script parallelizes using slurm arrays (see https://docs.rc.fas.harvard.edu/kb/submitting-large-numbers-of-jobs/). FASRC likes these more, and we're being nice to FASRC.
  - In the file, `#SBATCH -a 1-100` submits jobs with array IDs 1 through 100, i.e., 100 replications of the job. Tweak this line if you want to run more or fewer replications.
- As written, the slurm script outputs `.out` and `.err` files to `logs/simARRAYID.out` and `logs/simARRAYID.err`. This means two things:
  1. You need to make a logs directory so the script doesn't complain
  2. This overwrites logs from new jobs (since the files are only indexed by ARRAYID not JOBID). I like this because then I don't have to go and delete old logs, but be aware of this when debugging.
- To fix the error `Batch script contains DOS line breaks (\r\n) instead of expected UNIX line breaks (\n)`: run `dos2unix run_it.slurm` in the cluster terminal

Installing packages on the cluster

There might be better ways to install packages now (see https://docs.rc.fas.harvard.edu/kb/rstudio-server-vs-rstudio-desktop/), but this is the simple way I know how to do it.

1. If you don't have it already, under your home directory, make the directory `apps/R-4.2.2` (or whatever version of R the cluster is on, see https://docs.rc.fas.harvard.edu/kb/all-modules/)
2. Start an interactive R session: on the command line, run:

```
srun -p test --pty -t 60 -n1 --mem 4000  /bin/bash
```

3. Load the most recent R module (see https://docs.rc.fas.harvard.edu/kb/all-modules/):

```
module load R/4.2.2-fasrc01
```

4. Mark folder that will contain all of your installed packages:

```
export R_LIBS_USER=$HOME/apps/R-4.2.2:$R_LIBS_USER
```

5. Run R: type `R` in the terminal and run it
6. Now you're within R, so install packages as usual. See `install_needed_packages.R` to find a (potentially non-comprehensive) list of the packages I needed to install to make things work.